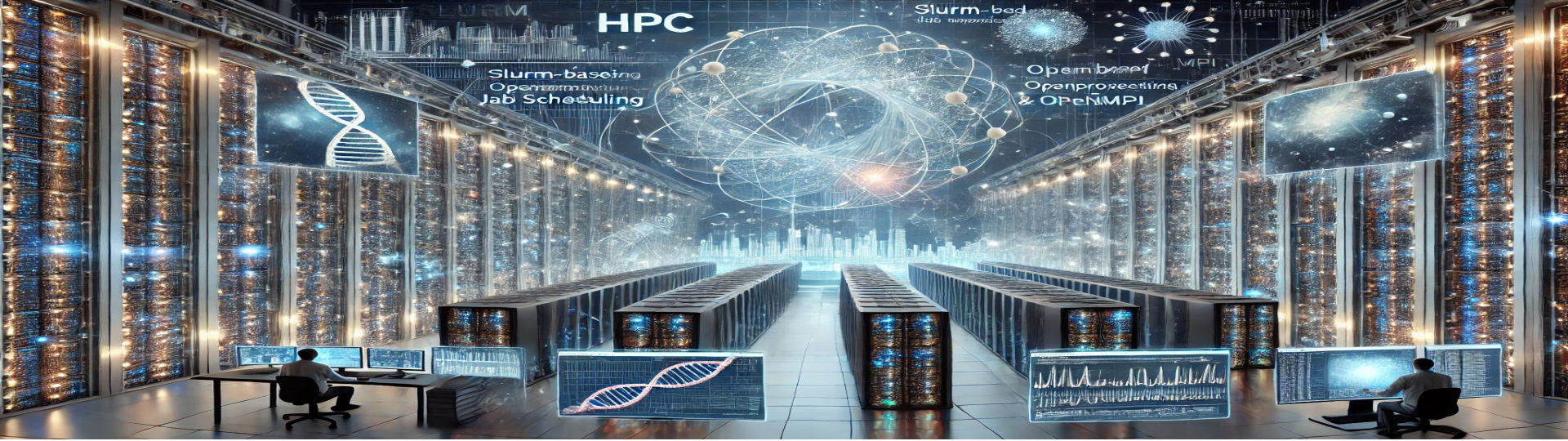


ilifu Online Training

Session 3: Parallelism

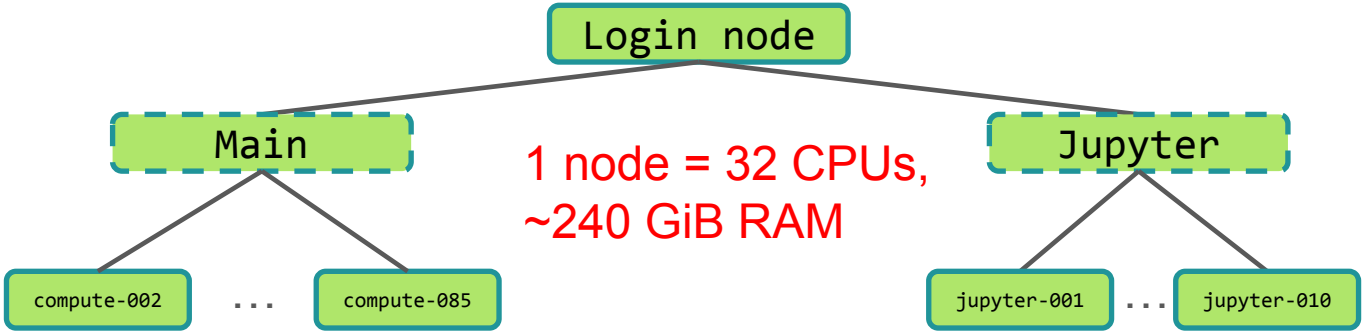
16 September 2025

Dane Kennedy, ilifu Bioinformatics Support
University of Cape Town

SLURM

http://docs.ilifu.ac.za/#/getting_started/submit_job_slurm

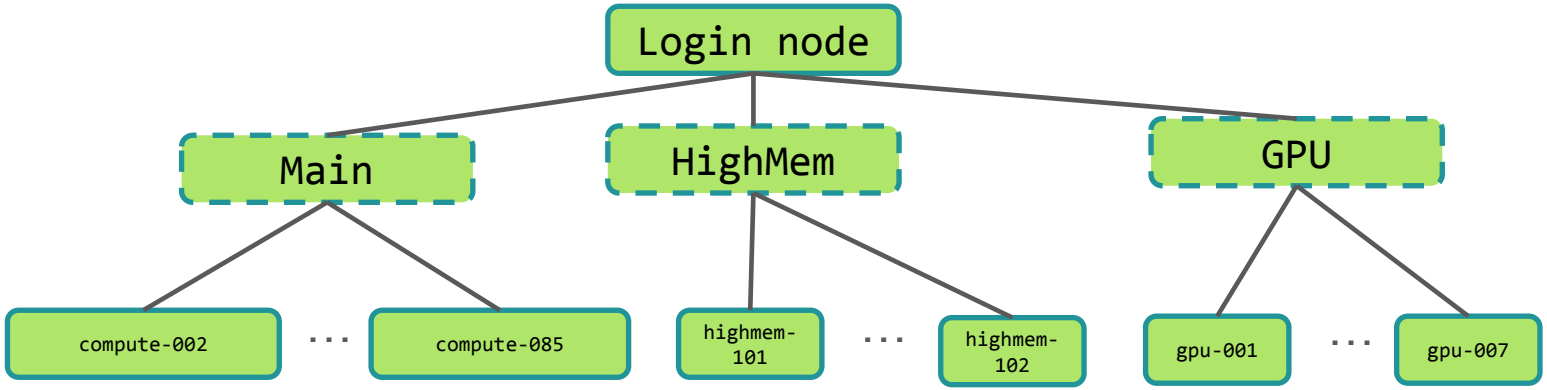
- Login node (job submission & management)
  - Where you land when you log in (also known as “head node”)
  - **Run SLURM commands/submit jobs, but not software/heavy processes**
- Compute nodes
  - Where your processes run (also known as “worker nodes”)
  - Via modules /Singularity containers



SLURM

http://docs.ilifu.ac.za/#/tech_docs/running_jobs?id=_4-specifying-resources-when-running-jobs-on-slurm

- Partitions (other than Jupyter) – see with ‘`sinfo`’:
  - `Main + Devel`: 85ish nodes, each w/ 32 CPUs, ~240 GB (usable) RAM
  - `HighMem`: 2 nodes, w/ 32 CPUs, 503 GB (usable) RAM + 96 CPU 1.5 TB RAM
  - `GPU`: 7 nodes (P100, V100,..), each w/ 24-48 CPUs, 232-354 GB (usable) RAM



Parallelism

https://en.wikipedia.org/wiki/Parallel_computing

- Oxford definition for parallel processing
  - *a mode of operation in which a process is split into parts, which are executed simultaneously on different processors attached to the same computer [or different computers attached to the same cluster].*
  - A cluster includes many connected nodes, each with its own RAM & CPUs
  - A node = single computer / server / VM / machine / box
- The work is partitioned into smaller jobs, sometimes with a partition of the dataset

What is a program?

https://en.wikipedia.org/wiki/Computer_program

- Set of discrete instructions
- Carried out sequentially
- Example: print average grade of a class

```
1.  total = 0

2.  for grade in grades:
      total = total + grade

3.  average = total / number_of_grades

4.  print(average)
```

Parallel execution of a program

https://en.wikipedia.org/wiki/Divide-and-conquer_algorithm

- Partition grades into *n* sets and do the following:

```
1.  total = 0
2.  for grade in 1/n
    grades:

    total = total + grade

1.  average_1 = total /
    number_of_grades
```

...

```
1.  total = 0
2.  for grade in 1/n
    grades:

    total = total + grade

1.  average_n = total /
    number_of_grades
```

- Combine results

$$\text{average} = \frac{(\text{average_1} + \ldots + \text{average_n})}{\text{number_of_partitions}}$$

Parallelism

- Executing portions of program simultaneously
- Possible when we have many processors (cores/CPUs)
- Capacity dependent on structure of both hardware AND software
- Requires overall control/coordination mechanism
  - i.e. message passing in MPI / threading / OpenMP

Parallelism on the ilifu cluster

- A cluster includes many connected nodes
- Each node has RAM and multiple cores
- Some nodes have GPUs
- Work of job is partitioned into smaller jobs
- Sometimes with a partition of the data

Parallelism

- Can be achieved on a single machine / node
  - Distributes work over many CPUs
  - Typically implemented using threads / OpenMP
  - GPU

- Or over multiple machines / nodes
  - Distributes work over many tasks, over 1+ nodes
  - Each given amount of memory to use
  - Generally requires a cluster
  - Requires a message passing interface (MPI) wrapper
    - mpirun, srun (SLURM), mpicasa (CASA 5)
    - Version of wrapper outside and inside container / venv must match

- Hybrid parallelism? (MPI + OpenMP / MPI + GPU / ...)

- Managed on ilifu by SLURM

Parallelism

- Implementing a normal job in SLURM
  - Will only use 1 CPU, 1 task, and 1 node
  - Default for many processes
- Implementing a threading / OpenMP job in SLURM
  - Need to use >1 CPU, while nodes & tasks must be 1 (unless also using MPI)
    - `cpus-per-task` (not inherited from #SBATCH)
    - May need to `export OMP_NUM_THREADS`
- Implementing an MPI job in SLURM
  - Need to use >1 task, while nodes and CPUs can be 1
    - nodes, `ntasks-per-node`, `cpus-per-task`
    - Best to wrap singularity in MPI call
- Cannot exceed 32 CPUs (or tasks) per node

SLURM parameters

https://docs.ilifu.ac.za/#/getting_started/submit_job_slurm?id=customising-your-job-using-sbatchsrun-parameters

```
--nodes=              # the number of nodes allocated to job
--cpus-per-task=      # number of cpus per task
--mem-per-cpu=        # memory per cpu
--mem=                # memory per node
--ntasks-per-node=    # number of tasks per node
```

SLURM – serial and multi-CPU jobs

- If code is serial, i.e. doesn't use OpenMP or MPI, increasing CPUs or nodes will not decrease execution time

```
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1

python myscript.py
```

- Using multiple CPUs within a node with OpenMP, where N is an optional number of CPUs (utilised by myscript.py)

```
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=N
#SBATCH --mem-per-cpu=XGB
export OMP_NUM_THREADS=N
python myscript.py
```

- *Note: The maximum number of CPUs per node (32) will not always give the maximum speedup*

SLURM – multi-task jobs

- Can also specify tasks or tasks per node

```
#SBATCH --ntasks=N
#SBATCH --cpus-per-task=1
#SBATCH --mem=XGB

module add openmpi
mpirun python myscript.py
```

- Above example doesn't require knowledge of number of node's CPUs; below one does

```
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=N
#SBATCH --cpus-per-task=1
#SBATCH --mem=XGB

module add openmpi
mpirun python myscript.py
```

SLURM – multi-task and multi-CPU jobs

- Using multiple nodes with MPI

```
#SBATCH --nodes=N
#SBATCH --ntasks-per-node=n
#SBATCH --cpus-per-task=1
#SBATCH --mem=XGB
module add openmpi
mpirun python myscript.py
```

- *Note: Need to consider that internode communication is slower than intranode communication*

- `--mem` is memory per node, so N times XGB allocated overall (usable by some software)

- Using multiple nodes with MPI as well as multiple cores within node with OpenMP

```
#SBATCH --ntasks=N
#SBATCH --cpus-per-task=n
module add openmpi
mpirun python myscript.py
```

Live Demo time

https://github.com/ilifu/ilifu_user_training/tree/main/session3/tutorial1

Best practices

https://docs.ilifu.ac.za/#/getting_started/best_practices

- Don't run software / heavy processes / scp on the login node
  - Only submit jobs and run SLURM commands (sbatch, srun, squeue, etc)
  - Use transfer.ilifu.ac.za to transfer data (external/internal), not login node
- Before running a large job, identify the available resources
  - Use sinfo. Don't hog the cluster. Reduce your allocation if possible
  - Increase likelihood of jobs running with less memory and less walltime
- Use sbatch (srun / screen / tmux / mosh are volatile)
- Cleanup files that aren't needed
  - Old raw data, temporary products, /scratch data, etc
- Don't place large files in your home directory (/users)
- Use Singularity (you cannot install software on the nodes)

THANK YOU

Acknowledgements

Dr Jordan Collier for the original slides.

IDIA Inter-University Institute for Data Intensive Astronomy

UNIVERSITY OF CAPE TOWN
IYUNIVESITHI YASEKAPA • UNIVERSITEIT VAN KAAPSTAD

CBIO Computational Biology @ UCT